

APOSTILA PRÁTICA

Classificação com KNN

Guia explicativo do notebook `consulta_knn.ipynb`

Treinamento com a base de clientes e consulta de um novo cliente

Objetivo central: Entender como os dados são preparados, como o modelo KNN é treinado e como uma nova observação recebe uma classificação.

Material didático • 2026

Como usar esta apostila

A apostila acompanha a ordem das células do notebook. Execute cada célula no Jupyter Notebook, JupyterLab ou Google Colab e compare a saída com as explicações deste material.

1. Coloque `consulta_knn.ipynb`, `frutas_knn.csv` e `clientes_knn.csv` na mesma pasta.
2. Abra o notebook e execute as células de cima para baixo.
3. Na parte de clientes, altere os valores de `novo_cliente` para testar outros perfis.
4. Observe a classe prevista e as probabilidades retornadas pelo modelo.

Dependências: Python 3 com pandas, matplotlib e scikit-learn. Em um ambiente novo, instale-os com:
`pip install pandas matplotlib scikit-learn`

Arquivos utilizados

- `consulta_knn.ipynb` — notebook com todo o fluxo de análise e classificação.
- `clientes_knn.csv` — 180 clientes sintéticos, divididos igualmente em segmentos baixo, médio e alto.
- `frutas_knn.csv` — 150 frutas sintéticas, divididas em maçã, laranja e pera.

Mapa rápido do fluxo

Etapa	Comando principal	Resultado
Carregar	<code>pd.read_csv(...)</code>	DataFrame com os clientes
Separar	<code>train_test_split(...)</code>	Treino e teste estratificados
Normalizar	<code>StandardScaler()</code>	Atributos na mesma escala
Treinar	<code>modelo.fit(X_train, y_train)</code>	KNN aprende com exemplos
Avaliar	<code>modelo.predict(X_test)</code>	Previsões para dados reservados
Consultar	<code>modelo.predict(novo_cliente)</code>	Segmento previsto

1. O que é o algoritmo KNN?

KNN significa K-Nearest Neighbors, ou K Vizinhos Mais Próximos. Para classificar um novo registro, o algoritmo calcula quais exemplos conhecidos estão mais próximos e adota a classe mais frequente entre os k vizinhos.

Exemplo: Se $k = 5$ e, entre os cinco clientes mais próximos, quatro pertencem ao segmento alto e um ao segmento médio, a previsão será alto.

Por que normalizar os atributos?

O KNN usa distância. Renda mensal pode estar na casa dos milhares, enquanto compras por mês pode variar de 0 a 15. Sem normalização, a renda dominaria o cálculo apenas por possuir números maiores.

```
Pipeline([
    ('escala', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=k))
])
```

StandardScaler() — centraliza cada atributo em torno da média e ajusta sua escala pelo desvio-padrão.

KNeighborsClassifier(...) — cria o classificador KNN com a quantidade escolhida de vizinhos.

Pipeline(...) — garante que a normalização e o KNN sejam aplicados sempre na ordem correta.

O significado de k

- k muito pequeno: o modelo pode ficar sensível a ruídos e casos isolados.
- k muito grande: classes diferentes podem ser excessivamente misturadas.
- O notebook testa valores ímpares de 1 a 15 e seleciona o de maior acurácia.

2. Bibliotecas e localização dos arquivos

```
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay, classification_report
```

Path — trabalha com caminhos de arquivos de modo portátil.

pandas — lê e manipula as tabelas CSV por meio de DataFrames.

matplotlib.pyplot — gera os gráficos de acurácia e matriz de confusão.

train_test_split — separa exemplos para treinamento e avaliação.

Pipeline — encadeia normalização e classificação.

classification_report — mostra precisão, revocação e F1-score por classe.

Detecção automática da pasta

```
BASE_DIR = Path.cwd()
if not (BASE_DIR / 'frutas_knn.csv').exists():
    BASE_DIR = BASE_DIR / 'basedados'
print('Pasta das bases:', BASE_DIR.resolve())
```

O código primeiro procura os CSVs na pasta atual. Se não encontrar `frutas_knn.csv`, acrescenta `basedados` ao caminho. Isso permite abrir o notebook tanto de dentro da pasta `basedados` quanto a partir da pasta principal do projeto.

Atenção: Se os arquivos estiverem em outro local, altere `BASE_DIR` manualmente para apontar para a pasta correta.

3. Exemplo inicial: base de frutas

A primeira base apresenta o fluxo completo de classificação antes de repeti-lo para clientes. Ela utiliza peso, diâmetro, teor de açúcar, firmeza e índice de cor para prever a classe da fruta.

```
frutas = pd.read_csv(BASE_DIR / 'frutas_knn.csv')
display(frutas.head())
print(frutas.shape)
display(frutas['classe_fruta'].value_counts())
```

pd.read_csv(...) — carrega o CSV em um DataFrame chamado frutas.

head() — exibe as cinco primeiras linhas para conferência.

shape — informa a quantidade de linhas e colunas.

value_counts() — conta quantos exemplos existem em cada classe.

```
atributos_frutas = ['peso_g', 'dimetro_cm', 'teor_acucar_brix',
                    'firmeza_kgf', 'indice_cor']
X = frutas[atributos_frutas]
y = frutas['classe_fruta']
```

A variável X recebe as entradas usadas para calcular distâncias. A variável y recebe a resposta correta que o modelo deverá aprender a prever. A coluna id é propositalmente ignorada, pois identifica a linha, mas não descreve a fruta.

Regra importante: Identificadores geralmente não devem entrar no treinamento. Eles podem criar padrões artificiais sem significado real.

4. Base de clientes: carregamento e inspeção

```
clientes = pd.read_csv(BASE_DIR / 'clientes_knn.csv')
display(clientes.head())
print(clientes.shape)
display(clientes['segmento'].value_counts())
```

Esse trecho cria o DataFrame clientes. A inspeção inicial confirma os nomes das colunas, o tamanho da base e o equilíbrio entre os segmentos.

Colunas da base

Coluna	Interpretação
idade	Idade do cliente, em anos.
renda_mensal	Renda mensal estimada.
compras_mes	Quantidade média de compras no mês.
ticket_medio	Valor médio gasto por compra.
visitas_site_mes	Quantidade de visitas ao site no mês.
dias_desde_ultima_compra	Recência: menor valor indica compra mais recente.
segmento	Classe-alvo: baixo, médio ou alto.

Leitura de negócio: Um cliente do segmento alto tende a combinar maior renda, mais compras, ticket maior, mais visitas e menor tempo desde a última compra. O KNN considera a combinação de todos esses fatores.

5. Preparação dos dados de clientes

```
atributos_clientes = [  
    'idade', 'renda_mensal', 'compras_mes', 'ticket_medio',  
    'visitas_site_mes', 'dias_desde_ultima_compra'  
]  
X = clientes[atributos_clientes]  
y = clientes['segmento']
```

Aqui são definidos os atributos de entrada e a classe que será prevista.

`clientes[atributos_clientes]` — seleciona somente as seis colunas explicativas e cria X.

`clientes['segmento']` — seleciona a resposta conhecida e cria y.

Separação entre treino e teste

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42, stratify=y  
)
```

`test_size=0.25` — reserva 25% dos registros para testar o modelo; os 75% restantes são usados no treinamento.

`random_state=42` — fixa o sorteio, tornando os resultados reproduzíveis.

`stratify=y` — preserva aproximadamente a mesma proporção de segmentos no treino e no teste.

Por que separar?: Avaliar com os mesmos clientes usados no treinamento produziria uma estimativa otimista. O conjunto de teste simula clientes ainda não vistos.

Dimensões esperadas

Com 180 registros e teste de 25%, aproximadamente 135 clientes ficam no treino e 45 no teste. A estratificação mantém os três segmentos representados.

6. Treinamento e escolha do melhor k

```
resultados_k = []
for k in range(1, 16, 2):
    modelo = Pipeline([
        ('escala', StandardScaler()),
        ('knn', KNeighborsClassifier(n_neighbors=k))
    ])
    modelo.fit(X_train, y_train)
    resultados_k.append({
        'k': k,
        'acuracia': modelo.score(X_test, y_test)
    })
```

O que acontece em cada repetição

5. range(1, 16, 2) gera os valores 1, 3, 5, 7, 9, 11, 13 e 15.
6. Pipeline cria uma nova sequência de normalização e KNN para o k atual.
7. fit ajusta o normalizador usando apenas o treino e armazena os exemplos de treinamento para o KNN.
8. score prevê o conjunto de teste e calcula a proporção de acertos.
9. append guarda o k e sua acurácia para comparação.

Treinamento no KNN: Diferentemente de modelos que estimam muitos coeficientes, o KNN essencialmente guarda os exemplos de treino. Na consulta, ele calcula distâncias até esses exemplos.

```
resultados_clientes = pd.DataFrame(resultados_k)
display(resultados_clientes)
```

O DataFrame resultados_clientes permite identificar qual valor de k obteve a maior acurácia no conjunto de teste.

7. Modelo final de clientes

```
melhor_k_clientes = int(
    resultados_clientes.loc[
        resultados_clientes['acuracia'].idxmax(), 'k'
    ]
)
```

`idxmax()` — retorna o índice da linha que possui a maior acurácia.

`.loc[... , 'k']` — obtém o valor de k armazenado naquela linha.

`int(...)` — converte o resultado para um número inteiro aceito pelo classificador.

```
modelo_clientes = Pipeline([
    ('escala', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=melhor_k_clientes))
])
modelo_clientes.fit(X_train, y_train)
```

Esse é o treinamento definitivo. O pipeline usa o k vencedor e recebe os dados de treinamento. A variável `modelo_clientes` deve ser preservada, pois será usada para avaliar e consultar novos clientes.

Comando principal: `modelo_clientes.fit(X_train, y_train)` é a instrução que treina o modelo final com os atributos e segmentos conhecidos.

O que o pipeline aprende

- A média e o desvio-padrão de cada atributo, calculados somente no treino.
- Os exemplos normalizados e seus segmentos, que servirão como vizinhos.
- A quantidade `melhor_k_clientes` de vizinhos que participarão da votação.

8. Avaliação do modelo

```
pred = modelo_clientes.predict(X_test)
print('Melhor k:', melhor_k_clientes)
print('Acurácia:', accuracy_score(y_test, pred))
print(classification_report(y_test, pred))
ConfusionMatrixDisplay.from_predictions(y_test, pred, cmap='Greens')
plt.show()
```

predict(X_test) — classifica todos os clientes reservados para teste.

accuracy_score(y_test, pred) — calcula a fração total de previsões corretas.

classification_report(...) — apresenta métricas separadas para baixo, médio e alto.

ConfusionMatrixDisplay — mostra visualmente acertos e confusões entre as classes.

Como interpretar as métricas

- Precisão: entre os clientes previstos em uma classe, quantos realmente pertenciam a ela.
- Revocação: entre os clientes que realmente pertenciam à classe, quantos foram encontrados.
- F1-score: equilíbrio entre precisão e revocação.
- Support: quantidade de exemplos reais da classe no teste.

Cuidado: A base é sintética e didática. Uma acurácia alta não significa que o mesmo desempenho ocorrerá com clientes reais. Dados reais exigem validação, revisão de vieses e acompanhamento contínuo.

9. Consulta de um cliente novo

Parte principal: Este é o trecho usado após o treinamento para descobrir o segmento provável de um cliente que não estava na base.

```
novo_cliente = pd.DataFrame(
    [[40, 7200, 6, 230, 7, 24]],
    columns=atributos_clientes
)
```

A linha representa um cliente com os valores abaixo, exatamente na ordem definida em atributos_clientes:

Atributo	Valor	Leitura
idade	40	40 anos
renda_mensal	7200	R\$ 7.200 por mês
compras_mes	6	6 compras mensais
ticket_medio	230	R\$ 230 por compra
visitas_site_mes	7	7 visitas mensais
dias_desde_ultima_compra	24	Última compra há 24 dias

Ordem obrigatória: Os valores e os nomes das colunas devem corresponder aos atributos usados no treinamento. Usar nomes ou ordem diferentes pode produzir erro ou interpretação incorreta.

10. Previsão e probabilidades

```
segmento = modelo_clientes.predict(novo_cliente)[0]
probabilidades = pd.Series(
    modelo_clientes.predict_proba(novo_cliente)[0],
    index=modelo_clientes.classes_
)
print('Segmento previsto:', segmento)
display(
    probabilidades.sort_values(ascending=False)
    .rename('probabilidade')
)
```

predict(novo_cliente) — normaliza o registro, encontra os vizinhos e retorna a classe escolhida pela maioria.

[0] — retira o primeiro resultado do vetor, pois foi consultado apenas um cliente.

predict_proba(...) — calcula a proporção de vizinhos que votou em cada segmento.

modelo_clientes.classes_ — fornece os nomes das classes na ordem das probabilidades.

sort_values(ascending=False) — ordena da maior para a menor probabilidade.

Interpretação da saída

Se o resultado for, por exemplo, segmento médio com probabilidade 0,80, isso significa que 80% dos k vizinhos consultados pertenciam ao segmento médio. Em um KNN com $k = 5$, esse valor corresponderia a quatro dos cinco vizinhos.

Probabilidade no KNN: A probabilidade é uma proporção de votos dos vizinhos, não uma garantia estatística de que o cliente pertence à classe.

Consultar vários clientes

```
novos_clientes = pd.DataFrame([
    [24, 2500, 2, 80, 2, 70],
    [42, 6500, 5, 190, 6, 30],
    [51, 13000, 10, 420, 11, 8]
], columns=atributos_clientes)

novos_clientes['segmento_previsto'] = modelo_clientes.predict(novos_clientes)
display(novos_clientes)
```

11. Alterando o perfil consultado

Para testar outro cliente, modifique somente os seis números da lista e mantenha `columns=atributos_clientes`.

```
novo_cliente = pd.DataFrame(  
    [[IDADE, RENDA, COMPRAS, TICKET, VISITAS, RECENCIA]],  
    columns=atributos_clientes  
)
```

Exemplos para experimentação

```
# Perfil com menor atividade recente  
[[22, 2200, 1, 60, 1, 90]]  
  
# Perfil intermediário  
[[37, 5600, 5, 175, 5, 32]]  
  
# Perfil de alto valor e alta atividade  
[[48, 14500, 11, 450, 12, 5]]
```

Experimento: Altere um atributo de cada vez. Assim fica mais fácil perceber como renda, frequência, ticket e recência influenciam a vizinhança do cliente.

Valores coerentes

- Não use texto em colunas numéricas.
- Evite valores impossíveis, como idade negativa ou recência menor que zero.
- Perfis muito fora das faixas da base podem receber uma classe, mas a previsão será uma extrapolação pouco confiável.
- Em aplicações reais, valide dados ausentes e limites antes de chamar `predict`.

12. Erros comuns e como resolver

Arquivo CSV não encontrado

```
FileNotFoundError: ... clientes_knn.csv
```

Confira o valor exibido em Pasta das bases e verifique se os três arquivos estão juntos. Se necessário, defina `BASE_DIR` diretamente.

```
BASE_DIR = Path(r'C:\caminho\para\basedados')
```

Biblioteca não instalada

```
ModuleNotFoundError: No module named 'sklearn'
```

Instale as dependências no mesmo ambiente Python usado pelo notebook.

```
pip install pandas matplotlib scikit-learn
```

Número diferente de atributos

O modelo foi treinado com seis atributos de clientes. A nova consulta também precisa ter seis valores e as mesmas colunas.

Modelo ainda não existe

Se aparecer `NameError` para `modelo_clientes`, execute primeiro as células de carregamento, preparação, escolha de `k` e treinamento.

Ordem de execução: Em notebooks, uma célula depende das variáveis criadas nas células anteriores. Reiniciar o kernel apaga essas variáveis.

13. Roteiro de atividade prática

10. Execute a inspeção da base clientes_knn.csv e confirme 180 registros.
11. Identifique quais colunas compõem X e qual coluna compõe y.
12. Execute a divisão treino/teste e registre os tamanhos de X_train e X_test.
13. Compare a acurácia dos diferentes valores de k.
14. Treine modelo_clientes usando melhor_k_clientes.
15. Analise a matriz de confusão e encontre as classes que mais se confundem.
16. Consulte os três perfis sugeridos e compare os segmentos previstos.
17. Crie seu próprio perfil e explique por que esperava determinado segmento.

Perguntas para discussão

- Por que a coluna id_cliente não deve ser usada como atributo?
- O que mudaria se StandardScaler fosse removido?
- Por que não devemos avaliar o modelo apenas com os dados de treino?
- Uma alta acurácia é suficiente para implantar o modelo em uma empresa?
- Quais riscos existem ao classificar clientes reais em segmentos?

14. Resumo dos comandos essenciais

```
# 1. Carregar
clientes = pd.read_csv(BASE_DIR / 'clientes_knn.csv')

# 2. Definir entradas e resposta
X = clientes[atributos_clientes]
y = clientes['segmento']

# 3. Separar treino e teste
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# 4. Criar e treinar
modelo_clientes = Pipeline([
    ('escala', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=melhor_k_clientes))
])
modelo_clientes.fit(X_train, y_train)

# 5. Criar um cliente novo
novo_cliente = pd.DataFrame(
    [[40, 7200, 6, 230, 7, 24]],
    columns=atributos_clientes
)

# 6. Consultar
segmento = modelo_clientes.predict(novo_cliente)[0]
print(segmento)
```

Sequência mínima: Carregar → selecionar atributos → separar → normalizar e treinar → criar novo cliente → prever.

Conclusão

O notebook demonstra um ciclo completo de aprendizado supervisionado. O modelo aprende com clientes cujo segmento é conhecido, é testado com registros separados e depois classifica novos perfis. O ponto central é manter o mesmo pré-processamento e a mesma estrutura de atributos em todas as etapas.

Ao avançar para dados reais, complemente o processo com validação cruzada, tratamento de valores ausentes, análise de vieses, métricas adequadas ao objetivo de negócio e monitoramento do desempenho ao longo do tempo.